

💡 OOPs kya hota hai?

- *Object Oriented Programming System*
- Code ko **real world objects** jaise sochke likhna
- Jaise: Car, Human, Animal — sab kuch ek *class* ban jaata hai 🤖

🔧 Constructor Kya Hota Hai?

Constructor ek special method hai jo class ka **object banate waqt** automatically call hota hai. **Object banate hi constructor apne aap run ho jata hai** — jaise kisi machine ka switch on karte hi wo kaam karna shuru kar de.

Main kaam constructor ka hota hai **object ki initial values set karna**. 🤖

Constructor ke Features:

- **Automatic:** Object banate hi constructor apne aap call ho jaata hai.
- **Initialization:** Object ke attributes ko **initial** (starting) values dena.
- `__init__()` method ka use hota hai constructor ke liye.

1. Class & Object

✓ English:

- **Class:** A blueprint/template to create objects.
- **Object:** A real-world entity created using a class.

🇮🇳 Hindi Touch:

Class ek design hai, object us design se bana item hai.

class Car:

```
def __init__(self, brand):
```

```
    self.brand = brand
```

```
car1 = Car("BMW")
```

```
print(car1.brand)
```

Output: BMW

🔧 2. Constructor (`__init__()` Method)

✓ English:

A special method that runs automatically when an object is created.

👤 Hindi Touch:

Object bante hi `__init__()` apne aap chal jaata hai — jaise welcome message ho

class Student:

```
def __init__(self, name):
```

```
    print("Hello", name)
```

```
s1 = Student("Rahul")
```

Output: Hello Rahul

👤 3. self Keyword

✓ English:

- Refers to the current object of the class.
- Used to access variables & methods inside the class.

👤 Hindi Touch:

Self ka matlab — *main object* khud

🏠 Real-life Example:

Soch, teri class mein 2 dost hain — **Ravi** aur **Aman**.

Dono ka apna **naam** hai, **age** hai, **marks** hain.

- Jab Ravi ko bolna hai, "Mera naam Ravi hai" → Ravi ki information chahiye.
- Jab Aman ko bolna hai, "Mera naam Aman hai" → Aman ki information chahiye.

Ab **Ravi aur Aman** ki details **alag hain**, right ?

Dono ko ek hi method (function) ke through print karwana hai. Lekin kaise?

Answer: `self` ka use karte hain! 🧐

class Student:

```
def __init__(self, name):
```

```
    self.name = name
```

→ # self.name = student ka naam

```
def show_name(self):
```

```
    print("Mera naam hai", self.name) → # ye wahi naam print karega jo self ke paas hoga
```

Ravi ka object

```
s1 = Student("Ravi")
```

→ # s1 object ka naam hai Ravi

Aman ka object

```
s2 = Student("Aman")
```

→ # s2 object ka naam hai Aman

```
s1.show_name()
```

→ # Output: Mera naam hai Ravi

```
s2.show_name()
```

→ # Output: Mera naam hai Aman

◆ Step-by-step samajh:

1. **Ravi ka object (s1) bana:**
 - o `self.name = Ravi` → Ravi ka naam is object mein store ho gaya.
2. **Aman ka object (s2) bana:**
 - o `self.name = Aman` → Aman ka naam is object mein store ho gaya.
3. Jab tu `s1.show_name()` call karta hai:
 - o `self` ko **s1** object milega, aur **"Mera naam hai Ravi"** print karega.
4. Jab tu `s2.show_name()` call karta hai:
 - o `self` ko **s2** object milega, aur **"Mera naam hai Aman"** print karega.

💡 `self` ko simple tarike se yaad karo:

- `self` ka matlab hai — **"ye jo object hai, wahi kaam kar raha hai."**
- Jab bhi koi method call hota hai, `self` batata hai kaunsa object us method ko call kar raha hai.

🔗 Short mein samjhne ka tareeka:

- Agar tu `s1.show_name()` call karta hai, to `self` ka matlab hoga **s1**.
- Agar tu `s2.show_name()` call karta hai, to `self` ka matlab hoga **s2**.

✳ Why `self` important?

- Agar tu `self` use nahi karega, to Python ko yeh nahi pata chalega ke kis object ka naam print karna hai.
- `self` object ko identify karta hai. Jaise **"Mujhe Ravi ka naam chahiye"** ya **"Mujhe Aman ka naam chahiye"**.

Python OOPs: Inheritance 🏠

💡 What is Inheritance?

Inheritance ka matlab hota hai:

"Ek class apne parent class ke features (variables + methods) ko use kar sakti hai."

🔗 Matlab **code reuse** karna, aur naye features add karna — bina pura code dobara likhe! 🔄
Jaise real life mein bachche apne maa-baap ki properties inherit karte hain 🧑🏫

🔥 Python me inheritance kaise likhte hain ?

```
class Parent:
    def show(self):
        print("Parent property!")
```

```
class Child(Parent):
    def display(self):
        print("Child property!")
```

→ # Child class inherits from Parent

```
obj = Child()
obj.show()
obj.display()
```

→ # Inherited method

→ # Own method

Types of Inheritance in Python:

1 Single Inheritance

★ One parent class, one child class

```
class Father:
    def house(self):
        print("Papa ka Ghar 🏠")
```

```
class Son(Father):
    def bike(self):
        print("Meri Bike 🏍️")
```

```
s = Son()
s.house()
s.bike()
```

Use: Jab ek class se dusri class kuch features use kar rahi ho.

2 Multilevel Inheritance

★ Parent → Child → Grandchild

```
class Grandfather:
    def land(self):
        print("Dadaji ki zameen 🌳")
```

```
class Father(Grandfather):
    def car(self):
        print("Papa ki car 🚗")
```

```
class Son(Father):
    def bike(self):

        print("Meri bike 🏍️")
```

```
s = Son()
s.land()
s.car()
s.bike()
```

3 Hierarchical Inheritance

★ One parent, multiple children

```
class Parent:
    def property(self):
        print("Papa ki property 🏠")
```

```
class Son(Parent):
    def scooty(self):
        print("Son ki scooty 🛵")
```

```
class Daughter(Parent):
    def laptop(self):
        print("Daughter ka laptop 💻")
```

```
s = Son()
d = Daughter()
```

```
s.property()
s.scooty()
```

```
d.property()
d.laptop()
```

4 Multiple Inheritance

★ One child, multiple parents

```
class Mother:
    def cook(self):
        print("Maa ka khana 🍲")
```

```
class Father:
    def drive(self):
        print("Papa ki driving 🚗")
```

```
class Child(Mother, Father):
```



```
def play(self):
    print("Child playing 🎮")
```

```
c = Child()
c.cook()
```

```
c.drive()
c.play()
```



Hybrid Inheritance

★ Mix of 1+ types (mostly multiple + multilevel)

```
class A:
    def featureA(self):
        print("Feature A")
```

```
class B(A):
    def featureB(self):
        print("Feature B")
```

```
class C(A):
    def featureC(self):
        print("Feature C")
```

```
class D(B, C):
    def featureD(self):
        print("Feature D")
```

```
d = D()
d.featureA()
d.featureB()
d.featureC()
d.featureD()
```

✓ Types of Inheritance (Simple English Definitions)

① Single Inheritance

👉 When one class inherits from only one parent class.

👉 Used when a child needs features from one parent only.

2 Multilevel Inheritance

☞ When a class inherits from a class, which itself inherits from another class.

🐾 Like a chain: Grandparent → Parent → Child.

3 Hierarchical Inheritance

☞ When multiple classes inherit from the same parent class.

🐾 One parent, many children.

4 Multiple Inheritance

☞ When a class inherits from more than one parent class.

🐾 One child, many parents.

5 Hybrid Inheritance

☞ A combination of two or more types of inheritance.

🐾 Used in complex situations with mixed structures.

🌀 POLYMORPHISM →

💬 Word Meaning:

"Poly" = Many

"Morph" = Forms

→ So, Polymorphism = One thing, many forms

★ Simple Bhasha Me:

Jab same naam ka function ya method, alag-alag objects ke liye alag kaam karta hai, use hi bolte hain Polymorphism 🧐

Type	Kya Hota Hai	Python Support
Overloading	Same method name, alag parameters	✗ Nahi direct
Overriding	Child class method overrides parent	✓ Yes

✓ Real Life Example:

Socho →

- **Human.run()** → Insaan daudta hai 🏃
- **Computer.run()** → Program chalta hai 💻
- **Fan.run()** → Fan ghoomta hai 🌀

Same word = run()

But sabka alag meaning → Yehi hai **Polymorphism**

1 Method Overriding (Run-time Polymorphism) ✓

class Animal:

```
def sound(self):
    print("Some sound 🐾")
```

```
class Dog(Animal):
    def sound(self):
        print("Bhow Bhow 🐶")
```

```
class Cat(Animal):
    def sound(self):
        print("Meow Meow 🐱")
```

Function to call

```
def make_sound(animal):
    animal.sound()
```

```
make_sound(Dog())    → # Output: Bhow Bhow
make_sound(Cat())
```

🔗 Yaha sound() naam same hai

But Dog ke liye kuch aur, Cat ke liye kuch aur

→ Polymorphism in action !

🕒 1 Compile Time Polymorphism (Method Overloading)

★ Definition:

→ Jab method ka decision compile karte waqt hi ho jaata hai, usse bolte hain **Compile Time Polymorphism**

🚫 Python me direct method overloading support nahi karta jaise Java/C++
But ek **jugaad** hoti hai - default arguments ya `*args`

class Calculator:

```
def add(self, a=0, b=0, c=0):
    print("Sum is:", a + b + c)
```

calc = Calculator()

calc.add(2, 3) → # Output: 5

calc.add(1, 2, 3) → # Output: 6

calc.add() → # Output: 0

Feature	Compile Time Polymorphism	Run Time Polymorphism
Also Known As	Method Overloading	Method Overriding
Decision Taken At	Compile Time	Run Time
Python Support?	Not direct (can mimic)	Fully Supported
Example Concept	<code>add(a, b)</code> vs <code>add(a, b, c)</code>	<code>Dog.speak()</code> vs <code>Cat.speak()</code>

*args in Python

Simple Definition:

*args ka matlab hota hai → "any number of arguments"

Isse hum ek function me **multiple values** pass kar sakte hain **without** fixing how many.

🔧 Why it's useful?

Python me method overloading **direct support** nahi karta 😞

Toh hum `*args` use karke ek function bana lete hain jo **multiple input** le sakta hai 🔄

✓ Example: Using *args to mimic Overloading

```
def add(*nums):  
    total = 0  
    for n in nums:  
        total += n  
    print("Sum is:", total)
```

```
add(2, 3)           # Output: Sum is: 5  
add(1, 2, 3)        # Output: Sum is: 6  
add(10)              # Output: Sum is: 10
```

🔒 Encapsulation in Python

📖 Simple English Definition:

Encapsulation means **hiding the internal details** of an object and **only showing what's necessary** 🧐

Jaise phone ka system andar complex hota hai, but tu sirf buttons use karta hai
→ Waise hi, class ke data ko **protect** karke rakhte hain using **private variables**

🔒 How to do Encapsulation in Python?

🔑 By making **variables private** using double underscore `__`

🔑 And using **getter/setter methods** to access/modify them

🔒 How Encapsulation Works in Python:

🔑 1. Private Variables using `__` (double underscore)

★ Matlab kya?

Jab tum kisi variable ke aage `__` **double underscore** laga deta hai...

```
self.__name = "Rahul"
```

...toh vo variable private ban jaata hai! 😊

→ Ab koi bahar se directly access nahi kar sakta ✗

EX:→

```
class Student:
    def __init__(self):
        self.__name = "Rahul"
s1 = Student()
print(s1.__name)           # ✗ ERROR aayega → AttributeError
👉 Kyunki __name private hai
```

Usko direct access nahi milta – Ye hi hota hai data hiding

✗ 2. Use Getter/Setter to Access/Change Private Variable

✓ Getter → Value get karta hai

✓ Setter → Value set/change karta hai

🔄 Code Example: →

```
class Student:
    def __init__(self):
        self.__name = "Rahul"

    def get_name(self):           # Getter
        return self.__name

    def set_name(self, new_name): # Setter
        self.__name = new_name

s1 = Student()

print(s1.get_name())           # ✓ Rahul
s1.set_name("Amit")            # ✓ change kiya
print(s1.get_name())           # ✓ Amit
```

: Simple Bhasha Me:

Tum __name ko hide kar raha hai (jaise locker me chhupa diya 🗝️)
Aur get_name() / set_name() se hi us locker ka lock khulta hai 🔓

- Data safe ✓
- Access controlled ✓
- Yehi hai Encapsulation ka asli kaam 🍷

Abstraction in Python

📄 Simple Definition:

Abstraction means **hiding complex code** and **showing only important stuff** to the user.

→ Jaise tu phone use karta hai, but andar ka code nahi dekhta 📱

💡 Abstraction = **Sirf zaroori cheezein dikhana, baaki ka logic chhupana**

🔒 Backend ka code → Hidden

✓ User ke saamne → Sirf interface (functions)

📁 Real-Life Example:

- Tum **ATM** pe paisa nikalta hai 💰
- Bas **amount daal ke confirm** karta hai
- Andar kya logic chal raha, kaise deduct ho raha... **tu nahi jaanta**
- Yehi hota hai **abstraction!**

🔧 In Python, abstraction karte hain:

✓ **Abstract Classes**

✓ **Abstract Methods**

(with `abc` module)

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC): # Abstract class
    @abstractmethod
    def sound(self): # Abstract method
        pass
```

```
class Dog(Animal):
    def sound(self):
        print("Barks 🐶")
```

```
class Cat(Animal):
    def sound(self):
        print("Meows 🐱")
```

```
d = Dog()
d.sound() # Output: Barks
```

```
c = Cat()
c.sound() # Output: Meows
```



```
d = Dog()
d.sound() # Barks
```

```
c = Cat()
c.sound() # Meows
```

main.py



Share

Run

Output

```
1 from abc import ABC, abstractmethod
2
3 class Animal(ABC): # Abstract class
4     @abstractmethod
5     def sound(self): # Abstract method
6         pass
7 class Dog(Animal):
8     def sound(self):
9         print("Barks 🐶")
10 class Cat(Animal):
11     def sound(self):
12         print("Meows 🐱")
13
14 d = Dog()
15 d.sound() # Output: Barks
16 c = Cat()
17 c.sound() # Output: Meows
18 d = Dog()
19 d.sound() # Barks
20 c = Cat()
21 c.sound() # Meows
22
```

Barks 🐶

Meows 🐱

Barks 🐶

Meows 🐱

=== Code Execution Success

💡 Important Points:

- Animal is an **abstract class** (cannot be directly used ☹)
- sound() is an **abstract method** (force karta hai child class ko override kare)
- Dog / Cat me method ka real meaning define hota hai

→ User ko sirf . sound() dikh raha

Andar ka logic alag-alag class me chhupa hai → That's abstraction !